

**VIETNAM NATIONAL UNIVERSITY – HOCHIMINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF ELECTRICAL ENGINEERING**



**MIDTERM REPORT
GREEN HOUSE PROJECT**

Submitted by

ID: Bùi Gia Bảo - ITITIU22019

Date Submitted:	10/05/2026
Date Performed:	28/4/2026
Course Instructor:	Vo Minh Thanh

GRADING GUIDELINE FOR PROJECT REPORT

Number	Content	Score	Comment
1	Format (max 9%)		
	- __Font type	Yes No	
	- __Font size	Yes No	
	- Lab title	Yes No	
	- __Page number	Yes No	
	- Table of contents	Yes No	
	- Header/Footer	Yes No	
	- List of figures (if exists)	Yes No	
	- List of tables (if exists)	Yes No	
	- Lab report structure	Yes No	
2	English Grammar and Spelling (max 6%)		
	- Grammar	Yes No	
	- Spelling	Yes No	
3	Data and Result Analysis (max 85%)		
Total Score			

Signature:

Date:

TABLE OF CONTENT

1. Introduction.....	5
1.1 Project Background.....	5
1.2 Problem Statement.....	5
1.3 Objectives.....	6
1.4 Scope of the Project.....	6
Figure 1: LPC1768 Kit.....	8
2. System Overview.....	8
2.1 System Block Diagram.....	8
2.2 Features of the System.....	8
Figure 2: Main menu display.....	10
Figure 3: UART terminal output.....	11
2.3 Hardware Components.....	11
Table 1.....	13
2.4 System Workflow.....	13
3. Hardware Design.....	13
3.1 LPC1768 Microcontroller.....	13
Table 2.....	14
3.2 Sensor Interface.....	14
3.3 Actuator Interface.....	15
Table 3.....	15
3.4 GLCD Interface.....	16
3.5 UART Interface.....	16
3.6 Joystick Interface.....	17
Table 4.....	17
3.7 Timer and Interrupt System.....	18
4. System Design and Implementation.....	19
4.1 Overall System Design.....	19
4.2 Software Architecture.....	19
Table 5.....	19
4.3 Sensor Monitoring System.....	20
4.4 Actuator Control System.....	20
4.6 Operating Modes.....	20
4.6.1 Manual Mode.....	20
4.6.2 Auto Mode.....	21
4.6.3 Timer Mode.....	21
4.7 UART Communication.....	22
4.8 GLCD User Interface.....	22
4.9 Synchronization and RTOS Management.....	23
Mutex Usage.....	23
Semaphore Usage.....	23
4.10 System Workflow.....	23
5. Menu System Design.....	23

5.1 Menu System Overview.....	23
5.2 Menu Navigation Control.....	24
Joystick Functions.....	24
Table 6.....	24
5.3 Menu Display Design.....	24
Display Features.....	24
5.4 Menu Display Design.....	25
Display Features.....	25
5.5 Manual Control Menu.....	25
Features.....	25
5.6 Threshold Adjustment Menu.....	25
5.7 Duration Adjustment Menu.....	26
Features.....	26
5.8 Timer Control Menu.....	26
Timer Features.....	26
Table 7.....	27
5.9 Mode Selection Menu.....	27
Available Modes.....	27
Table 8.....	27
5.10 Menu Refresh and Synchronization.....	27
5.11 Debouncing Implementation.....	28
6. Peripheral Interfaces and Control Logic.....	28
6.1 ADC Reading.....	28
ADC Polling Method.....	28
6.2 UART Communication.....	29
Table 9.....	29
UART TX Operation.....	30
UART RX Operation.....	30
UART Output Example.....	30
6.3 Timer Interrupt.....	31
Table 10.....	31
1-Second Interrupt.....	31
Real-Time Clock Update.....	32
7. Code.....	32

LIST OF FIGURES

Figure 1: LPC1768 Kit.....	8
Figure 2: Main menu display.....	10
Figure 3: UART terminal output.....	11

LIST OF TABLES

Table 1.....	12
Table 2.....	13
Table 3.....	15
Table 4.....	17
Table 5.....	19
Table 6.....	24
Table 7.....	26
Table 8.....	27
Table 9.....	29
Table 10.....	31

1. Introduction

1.1 Project Background

Greenhouse systems are widely used in modern agriculture to provide a controlled environment for plant growth. Environmental conditions such as temperature, soil moisture, and light intensity directly affect plant health and productivity. Traditionally, these conditions are monitored and adjusted manually, which requires continuous human supervision and may lead to inefficiency and inaccurate control.

With the development of embedded systems and automation technology, greenhouse monitoring and control can now be implemented automatically using microcontrollers, sensors, and actuators. Automated greenhouse systems are capable of continuously monitoring environmental conditions and performing control actions without constant human involvement. This improves efficiency, reduces labor cost, and helps maintain stable growing conditions for plants.

In this project, an embedded-based greenhouse control system was developed using the LPC1768 microcontroller and CMSIS-RTOS. The system reads sensor data through ADC channels, displays information on a GLCD, communicates through UART, and controls actuators using different operating modes such as Auto Mode, Manual Mode, and Timer Mode.

1.2 Problem Statement

Maintaining a stable greenhouse environment manually can be difficult and inefficient. Environmental conditions may change continuously throughout the day, requiring frequent

monitoring and adjustment. Manual systems may also respond slowly to environmental changes, causing unsuitable conditions for plant growth.

Several challenges exist in traditional greenhouse management systems:

- Continuous monitoring requires human effort.
- Environmental conditions may not be adjusted accurately.
- Manual operation is time-consuming.
- Lack of real-time monitoring reduces system efficiency.
- It is difficult to manage multiple environmental parameters simultaneously.

To overcome these limitations, an automated greenhouse control system is required. The proposed system should be capable of reading sensor values, making automatic decisions, controlling actuators, and allowing user interaction through a display and joystick interface.

1.3 Objectives

The main objective of this project is to design and implement a real-time greenhouse monitoring and control system using the LPC1768 microcontroller and RTOS.

The specific objectives of the project are:

- To read environmental sensor values using ADC channels.
- To display system information and menus on the GLCD.
- To implement UART communication for monitoring and debugging.
- To develop multiple operating modes:
 - Auto Mode
 - Manual Mode
 - Timer Mode
- To control actuators automatically based on sensor thresholds.
- To allow manual control of actuators using the joystick.
- To implement timer-based actuator control.
- To apply RTOS concepts such as:
 - Multithreading
 - Mutex
 - Semaphore
 - Task scheduling
- To create a user-friendly menu system for system configuration

1.4 Scope of the Project

This project focuses on the development of a greenhouse automation prototype using embedded system concepts. The system is implemented on the LPC1768 development board using CMSIS-RTOS in Keil uVision5.

The project includes the following features:

- Reading three sensor inputs using ADC:

- Temperature sensor
 - Moisture sensor
 - Light sensor
- Controlling three actuators represented by LEDs:
 - Heater
 - Sprinkler
 - Light system
- Displaying menus and sensor data on the GLCD.
- Using the joystick for menu navigation and parameter adjustment.
- Supporting UART communication with a PC terminal.
- Implementing timer interrupt functionality for real-time clock operation.
- Supporting three control modes:
 - Auto Mode
 - Manual Mode
 - Timer Mode

The project is limited to simulation and demonstration using LEDs instead of actual high-power devices such as pumps or motors. Wireless communication, cloud monitoring, and IoT functionality are not included in this implementation.

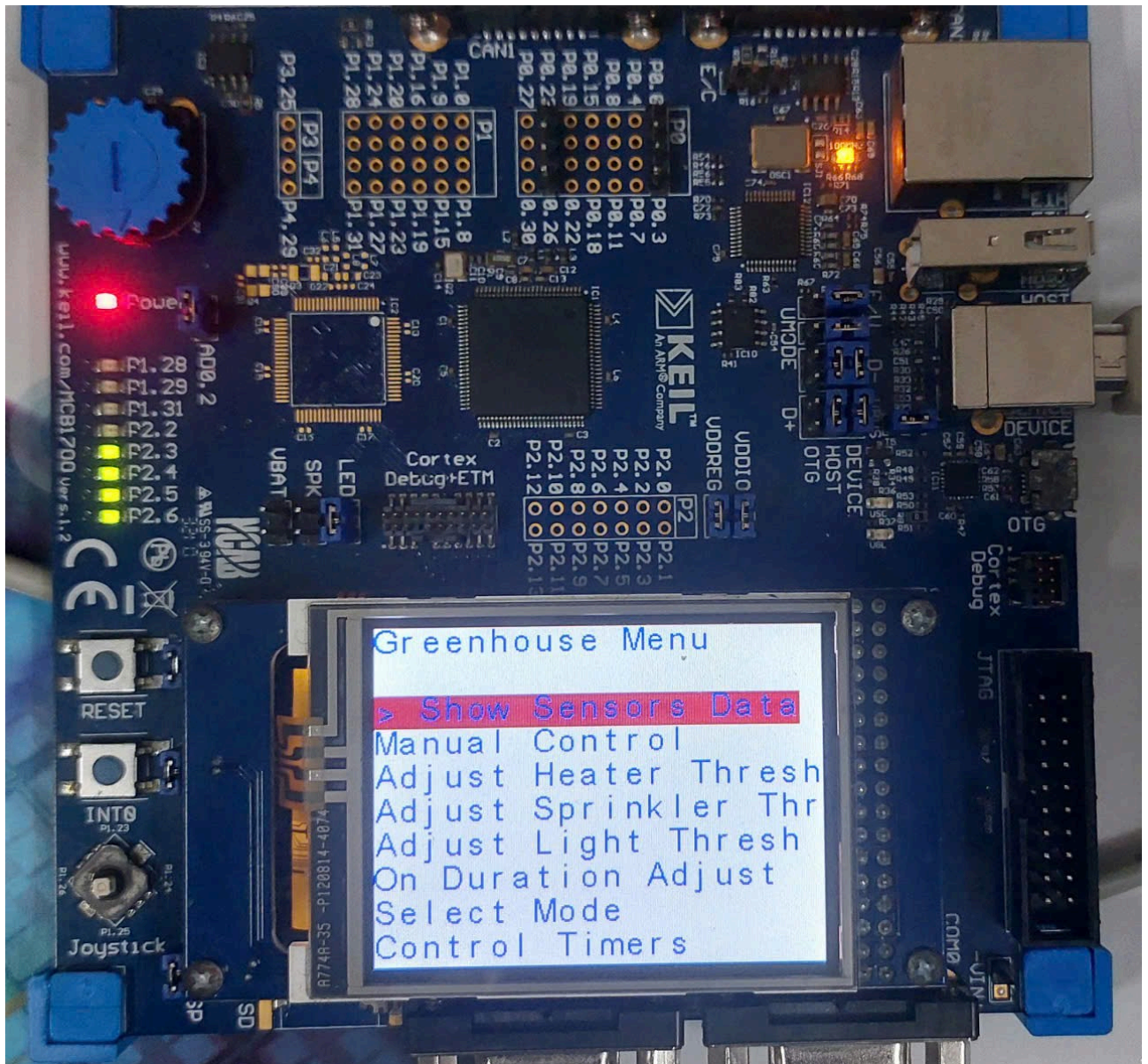


Figure 1: LPC1768 Kit

2. System Overview

2.1 System Block Diagram

The greenhouse control system is designed as an embedded real-time monitoring and control system. The LPC1768 microcontroller acts as the main processing unit responsible for sensor acquisition, actuator control, user interaction, and communication.

The system receives environmental data from multiple sensors through ADC channels. Based on the operating mode and predefined threshold values, the microcontroller controls actuators such as the heater, sprinkler, and lighting system. The GLCD is used to display system information and menus, while the joystick allows the user to navigate through

different control options. UART communication is also implemented for debugging and real-time monitoring through a serial terminal.

The overall system operation can be summarized as follows:

1. Sensors provide analog environmental data.
2. LPC1768 reads sensor values using ADC.
3. RTOS tasks process sensor data concurrently.
4. Control logic determines actuator behavior.
5. Results are displayed on the GLCD and UART terminal.
6. The user can configure settings through the joystick interface.

2.2 Features of the System

The greenhouse control system contains several important features that demonstrate real-time embedded system operation using RTOS.

Real-Time Sensor Monitoring

The system continuously reads sensor values using ADC channels. Environmental parameters such as temperature, moisture, and light intensity are updated periodically by RTOS threads.

Automatic Control Mode

In Auto Mode, the system compares sensor values with predefined threshold values. If the sensor value exceeds or falls below the threshold, the corresponding actuator is activated automatically.

Example:

- High temperature → Heater LED ON
- Low moisture → Sprinkler LED ON
- Low light intensity → Light LED ON

Manual Control Mode

In Manual Mode, the user can directly control the actuator states using the joystick through the GLCD menu system.

Timer Control Mode

The Timer Mode allows the user to configure a specific time for actuators to turn ON or OFF automatically using the internal timer system.

GLCD User Interface

The GLCD provides:

- Sensor data display
- Menu navigation
- Mode selection
- Timer configuration
- System status display

UART Communication

UART communication is used to:

- Display sensor readings
- Show debugging information
- Display system mode
- Monitor actuator status

RTOS Multitasking

The system uses CMSIS-RTOS to execute multiple tasks concurrently, improving responsiveness and system organization.

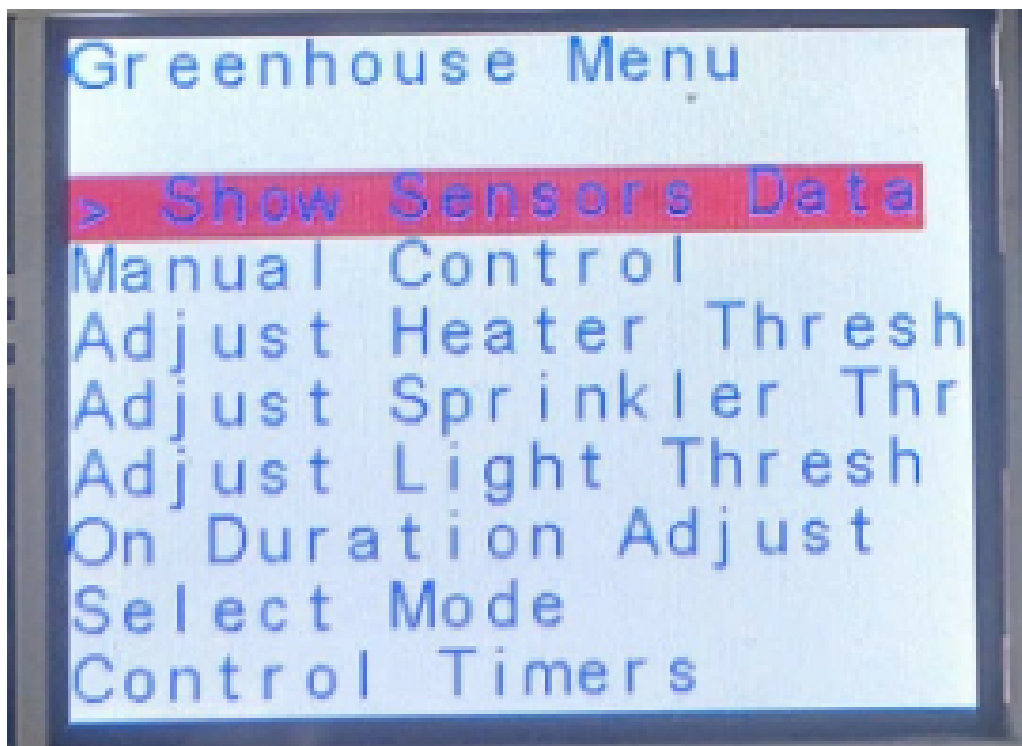


Figure 2: Main menu display

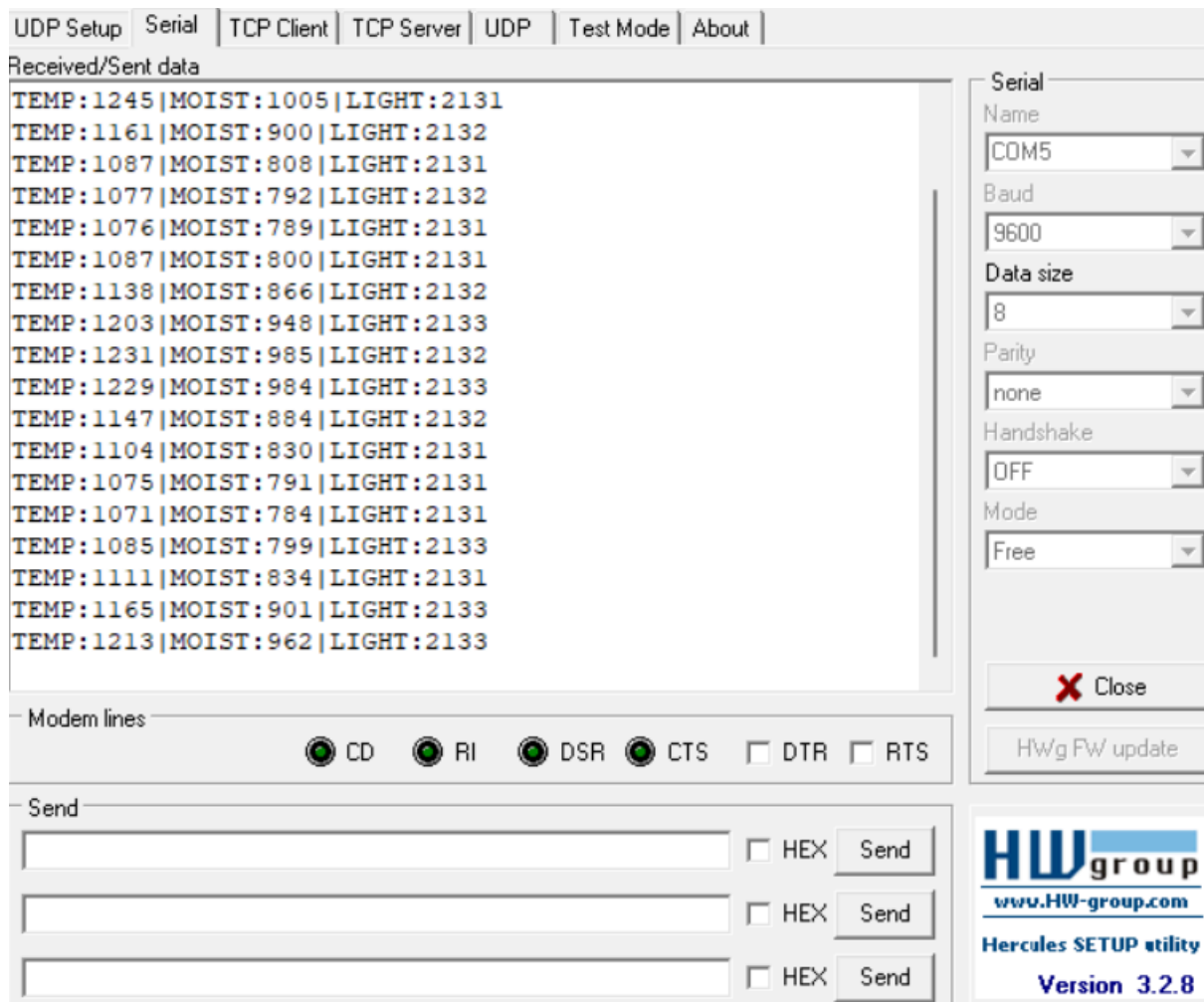


Figure 3: UART terminal output

2.3 Hardware Components

The greenhouse control system is implemented using several hardware components connected to the LPC1768 microcontroller board.

LPC1768 Microcontroller

The LPC1768 is an ARM Cortex-M3 based microcontroller used as the central controller of the system. It supports GPIO, ADC, UART, timers, and interrupt functions required for this project.

Sensors

Three ADC channels are used to simulate environmental sensors:

- Temperature sensor
- Moisture sensor
- Light sensor

Actuators

Three LEDs are used to simulate actuator devices:

- Heater
- Sprinkler
- Lighting system

GLCD Module

The GLCD is used to provide a graphical menu interface and display real-time system information.

Joystick

The joystick is used for:

- Menu navigation
- Selecting modes
- Editing timer values
- Confirming settings

UART Terminal

UART communication allows monitoring and debugging using software such as Hercules Serial Terminal.

Component	Function
LPC1768 Microcontroller	Main processing and control unit
GLCD Display	Displays menus and sensor information
Joystick	User input for menu navigation
UART Interface	Serial communication with PC
LEDs	Simulated actuators
ADC Inputs	Read analog sensor values
Timer1	Generates real-time clock interrupt

Table 1

2.4 System Workflow

The greenhouse control system operates continuously in real time under RTOS scheduling. Multiple threads work simultaneously to perform sensing, control, communication, and user interaction tasks.

The system workflow is summarized below:

1. System initialization
 - Initialize GPIO
 - Initialize ADC
 - Initialize UART
 - Initialize GLCD
 - Initialize Timer1
 - Start RTOS kernel
2. Sensor thread periodically reads ADC values.
3. Monitor threads compare sensor values with threshold values.
4. Depending on the selected mode:
 - Auto Mode:
Actuators respond automatically to sensor thresholds.
 - Manual Mode:
User manually controls actuators.
 - Timer Mode:
Actuators operate based on configured time values.
5. UART thread continuously sends system information to the serial terminal.
6. Menu thread handles joystick input and GLCD display updates.

3. Hardware Design

3.1 LPC1768 Microcontroller

The LPC1768 microcontroller is the main processing unit of the greenhouse control system. It is based on the ARM Cortex-M3 architecture and provides high performance with low power consumption. The microcontroller contains multiple peripherals such as GPIO, ADC, UART, timers, and interrupt controllers, which are required for real-time embedded applications.

In this project, the LPC1768 is responsible for:

- Reading sensor values through ADC channels
- Controlling actuator outputs
- Managing GLCD display operations

- Handling joystick input
- Executing RTOS tasks
- Performing UART communication
- Managing timer interrupts

The CMSIS-RTOS environment is used to organize multitasking operations efficiently on the microcontroller.

Important LPC1768 Peripherals Used

Peripheral	Purpose
GPIO	LED and joystick control
ADC	Sensor value acquisition
UART0	Serial communication
Timer1	Real-time clock interrupt
NVIC	Interrupt management

Table 2

3.2 Sensor Interface

The greenhouse system uses ADC channels to simulate environmental sensors. Analog values are converted into digital values by the internal 12-bit ADC of the LPC1768 microcontroller.

Three sensors are used:

- Temperature sensor
- Moisture sensor
- Light sensor

The ADC continuously samples sensor values using polling-based conversion.

ADC Initialization

```
void init_adc(void) {
```

```
    LPC_PINCON->PINSEL1 |= (1 << 14) | (1 << 16) | (1 << 18);
```

```
    LPC_SC->PCONP |= (1 << 12);
```

```
LPC_ADC->ADCR = (7 << 0) | (4 << 8) | (1 << 21);

}
```

The ADC configuration:

- Enables ADC power
- Selects ADC channels
- Configures ADC clock
- Powers ON the ADC module

ADC Reading Process

```
while (!(LPC_ADC->ADGDR & (1U << 31)));
```

3.3 Actuator Interface

The actuator system controls three output devices represented by LEDs in the prototype system.

Actuator	GPIO Pin
Heater	P1.29
Sprinkler	P1.31
Light System	P2.2

Table 3

The actuators operate differently depending on the selected system mode:

- Auto Mode
- Manual Mode
- Timer Mode

GPIO pins are configured as output pins to control the LEDs.

GPIO Initialization

```
LPC_GPIO1->FIODIR |= (1 << 29) | (1 << 31);
```

```
LPC_GPIO2->FODIR |= (1 << 2);
```

Actuator ON/OFF Control

```
actuators[type].port->FIOSET = actuators[type].pin;
```

```
actuators[type].port->FIOCLR = actuators[type].pin;
```

3.4 GLCD Interface

The GLCD module provides the graphical user interface for the greenhouse control system. It displays:

- Main menu
- Sensor values
- System mode
- Timer settings
- Actuator status

The GLCD improves user interaction and allows real-time monitoring directly on the hardware board.

GLCD Initialization

```
GLCD_Initialize();
```

```
GLCD_SetFont(&GLCD_Font_16x24);
```

Display Example

```
GLCD_DrawString(0, 0, "Greenhouse Menu");
```

3.5 UART Interface

UART communication is implemented to provide real-time serial monitoring and debugging. The UART terminal displays:

- Sensor values
- System modes
- Timer status
- Debugging messages

The UART communication uses UART0 at 9600 baud rate.

UART Initialization


```
LPC_UART0->LCR = 0x83;
```

```
LPC_UART0->DLL = 97;
```

```
LPC_UART0->LCR = 0x03;
```

UART Transmission Function

```
void send_uart_string(const char *str) {  
  
    while (*str) {  
  
        while (!(LPC_UART0->LSR & (1 << 5)));  
  
        LPC_UART0->THR = *str++;  
    }  
}
```

The UART transmission uses polling to check transmitter readiness before sending data.

3.6 Joystick Interface

The joystick is used as the primary user input device. It allows navigation through menus and adjustment of system parameters.

Joystick Functions

Direction	Function
UP	Move up / increase value
DOWN	Move down / decrease value
LEFT	Move left
RIGHT	Move right
CENTER	Confirm selection

Table 4

The joystick operates using GPIO input pins with pull-up resistors.

GPIO Configuration

```
LPC_GPIO1->FIODIR &= ~(JOYSTICK_UP_PIN |  
    JOYSTICK_DOWN_PIN |  
    JOYSTICK_CENTER_PIN |  
    JOYSTICK_LEFT_PIN |  
    JOYSTICK_RIGHT_PIN);
```

Joystick Reading Function

```
uint32_t read_joystick(void)
```

The function checks the state of each joystick direction and returns the corresponding input value.

Debouncing is implemented using software delay to reduce false triggering caused by mechanical switch noise.

3.7 Timer and Interrupt System

Timer1 is used to generate a periodic interrupt every one second. The interrupt updates the system clock used in Timer Mode.

Timer Initialization

```
LPC_TIM1->MR0 = SystemCoreClock - 1;
```

This configuration generates a 1 Hz interrupt frequency.

Timer Interrupt Handler

```
void TIMER1_IRQHandler(void)
```

The interrupt service routine:

- Clears interrupt flags
- Updates current time
- Supports timer scheduling

Time Update Function

```
current_time.seconds++;
```

The timer system provides:

- Real-time clock operation
- Timer-based actuator control
- Scheduled ON/OFF operation

4. System Design and Implementation

4.1 Overall System Design

The Greenhouse Control System was developed using the LPC1768 microcontroller with CMSIS-RTOS in Keil uVision5. The system combines multiple modules including sensors, actuators, LCD display, UART communication, joystick control, and timer scheduling into a single embedded application.

The system is divided into three operating modes:

- Manual Mode
- Auto Mode
- Timer Mode

Each mode controls the actuators differently depending on user settings and sensor conditions.

4.2 Software Architecture

The software was developed using RTOS multitasking architecture. Multiple threads run independently to improve responsiveness and system organization.

Main RTOS Threads

Thread	Function
sensor_thread	Read ADC sensor values
uart_thread	Send sensor data to UART
uart_receive_thread	Receive UART commands
monitor_thread	Check threshold conditions
control_thread	Control actuators
menu_thread	Handle GLCD menu system

Table 5

The RTOS uses:

- Mutexes for shared resources
- Semaphores for actuator synchronization
- Delays for task scheduling

4.3 Sensor Monitoring System

The system reads analog sensor values using the ADC peripheral. Sensor values are continuously updated by the sensor thread every 1 second.

Example ADC Reading Function

```
int read_adc_channel(uint8_t channel) {
    LPC_ADC->ADCR &= ~(0x7 << 0);
    LPC_ADC->ADCR |= (1 << channel);
    LPC_ADC->ADCR |= (1 << 24);

    while (!(LPC_ADC->ADGDR & (1U << 31)));

    return (LPC_ADC->ADGDR >> 4) & 0xFFF;
}
```

4.4 Actuator Control System

The actuators are represented using LEDs connected to GPIO pins.

Control Logic

- Heater ON when temperature exceeds threshold
- Sprinkler ON when moisture below threshold
- Light ON when light intensity below threshold

Example Control Code

```
if (adc_values[type] >= threshold_values[type]) {
    actuators[type].port->FIOSET = actuators[type].pin;
}
else {
    actuators[type].port->FIOCLR = actuators[type].pin;
}
```

4.6 Operating Modes

4.6.1 Manual Mode

In Manual Mode, the user directly turns actuators ON or OFF using joystick controls.

Features:

- User-controlled operation
- Independent LED control
- Real-time GLCD updates

4.6.2 Auto Mode

In Auto Mode, the system automatically compares sensor values with thresholds.

Features:

- Automatic decision making
- Threshold-based control
- Real-time monitoring

Auto Mode Algorithm

1. Read sensor value
2. Compare with threshold
3. Turn actuator ON/OFF
4. Repeat continuously

4.6.3 Timer Mode

Timer Mode allows actuators to operate according to user-defined schedules.

Users can:

- Select actuator
- Adjust hour/minute/second
- Save timer values
- Automatically trigger actuator

Timer Features

- Real-time clock counting
- Timer scheduling
- Edit mode and select mode
- UART notification messages

Example Timer Comparison

```
if (current_time.hours == timers[i].hours &&  
    current_time.minutes == timers[i].minutes &&  
    current_time.seconds == timers[i].seconds) {
```

```

        actuators[i].port->FIOSET = actuators[i].pin;
    }

```

4.7 UART Communication

UART is used for monitoring and debugging.

The system sends:

- Sensor values
- Mode changes
- Timer status
- System messages

Example UART Function

```

void send_uart_string(const char *str) {
    while (*str) {
        while (!(LPC_UART0->LSR & (1 << 5)));
        LPC_UART0->THR = *str++;
    }
}

```

4.8 GLCD User Interface

The GLCD provides a graphical menu system for user interaction.

Main menu options include:

- Show Sensor Data
- Manual Control
- Adjust Threshold
- Adjust Duration
- Control Timers
- Select Mode

The GLCD uses:

- Highlight selection
- Text-based navigation
- Real-time updates

4.9 Synchronization and RTOS Management

The system uses RTOS synchronization tools to prevent conflicts.

Mutex Usage

Mutexes protect:

- ADC shared data
- GLCD access

Example:

```
osMutexWait(adc_mutex, osWaitForever);  
osMutexRelease(adc_mutex);
```

Semaphore Usage

Semaphores synchronize actuator activation.

Example:

```
osSemaphoreRelease(actuator_sems[type]);
```

4.10 System Workflow

Overall system workflow:

1. Initialize hardware
2. Start RTOS kernel
3. Create threads
4. Read sensors continuously
5. Update GLCD
6. Process joystick input
7. Control actuators
8. Send UART messages

5. Menu System Design

5.1 Menu System Overview

The menu system is displayed on the GLCD screen and controlled using the joystick.

The main functions of the menu system include:

- Display sensor values
- Manual actuator control
- Threshold adjustment
- Duration adjustment
- Timer configuration
- Mode selection

5.2 Menu Navigation Control

The joystick is used as the input device for navigating through the menu.

Joystick Functions

Joystick Direction	Function
UP	Move upward / Increase value
DOWN	Move downward / Decrease value
LEFT	Move to previous field
RIGHT	Move to next field
CENTER	Select / Confirm

Table 6

5.3 Menu Display Design

The GLCD display updates dynamically based on user input.

Display Features

- Highlight selected menu item
- Clear text interface
- Real-time updates
- Mode indication
- Timer editing display

The selected menu item is highlighted using a gray background color.

Example Display Code

```
GLCD_SetBackgroundColor(COLOR_GRAY);
```



```
GLCD_DrawString(0, (selected_menu + 2) * 24, display_text);
```

5.4 Menu Display Design

The GLCD display updates dynamically based on user input.

Display Features

- Highlight selected menu item
- Clear text interface
- Real-time updates
- Mode indication
- Timer editing display

The selected menu item is highlighted using a gray background color.

Example Display Code

```
GLCD_SetBackgroundColor(COLOR_GRAY);  
GLCD_DrawString(0, (selected_menu + 2) * 24, display_text);
```

5.5 Manual Control Menu

The Manual Control menu allows direct user control of actuators.

Features

- Turn LEDs ON or OFF
- Select actuator using joystick
- Real-time actuator status display

Example Display

```
> Heater : ON  
Sprinkler : OFF  
Light : ON
```

The selected actuator is highlighted on the GLCD.

5.6 Threshold Adjustment Menu

This menu allows users to modify sensor threshold values used in Auto Mode.

Adjustment Method

- UP → increase value

- DOWN → decrease value
- CENTER → save and exit

Example Code

```
*actuators[type].threshold += 10;
```

5.7 Duration Adjustment Menu

The ON duration of each actuator can be modified by the user.

Features

- Configure actuator active time
- Adjustable in milliseconds
- Separate duration for each actuator

Example Display

Heater : 500 ms

Sprinkler : 700 ms

Light : 300 ms

5.8 Timer Control Menu

The timer menu allows users to configure actuator scheduling.

Timer Features

- Set hours, minutes, and seconds
- Select actuator
- Edit timer values
- Save timer configuration

The timer interface contains two modes:

Mode	Function
Select Mode	Choose actuator and field
Edit Mode	Modify timer value
Timer Navigation	
Action	Function
LEFT/RIGHT	Select hour/minute/second
UP/DOWN	Increase/decrease value
CENTER	Enter edit mode or save

Table 7

5.9 Mode Selection Menu

The Select Mode menu allows the user to choose system operation behavior.

Available Modes

Mode	Description
Manual Mode	User controls actuators
Auto Mode	Sensor-based automatic control
Timer Mode	Time-based scheduling

Table 8

5.10 Menu Refresh and Synchronization

The GLCD display must update continuously during navigation.

Mutex protection is used to prevent display conflicts between RTOS threads.

Example Mutex Usage

```

osMutexWait(glcd_mutex, osWaitForever);
osMutexRelease(glcd_mutex);

```

This ensures stable and synchronized screen updates.

5.11 Debouncing Implementation

Joystick buttons may generate repeated signals due to mechanical bouncing. Software debounce logic is implemented to prevent unwanted repeated actions.

Example Debounce Code

```
if (current_time - last_action >= DEBOUNCE_TIME_MS)
```

The debounce delay improves navigation stability and user experience.

6. Peripheral Interfaces and Control Logic

6.1 ADC Reading

The Analog-to-Digital Converter (ADC) is used to read analog sensor values such as temperature, moisture, and light intensity. The LPC1768 provides a 12-bit ADC module which converts analog voltages into digital values ranging from 0 to 4095.

In this project, the ADC operates using the polling method.

ADC Polling Method

The polling method continuously checks whether the ADC conversion has completed before reading the result.

ADC Polling Code

```
while (!(LPC_ADC->ADGDR & (1U << 31)));
```

Explanation

- **ADGDR** is the ADC Global Data Register.
- Bit 31 is the DONE flag.
- When conversion finishes, bit 31 becomes 1.
- The while loop keeps checking until conversion is complete.

ADC Reading Process

1. Select ADC channel
2. Start conversion
3. Wait until conversion finishes
4. Read ADC value

5. Store sensor data

Example ADC Function

```
int read_adc_channel(uint8_t channel) {  
  
    LPC_ADC->ADCR &= ~(0x7 << 0);  
  
    LPC_ADC->ADCR |= (1 << channel);  
  
    LPC_ADC->ADCR |= (1 << 24);  
  
    while (!(LPC_ADC->ADGDR & (1U << 31)));  
  
    return (LPC_ADC->ADGDR >> 4) & 0xFFF;  
}
```

6.2 UART Communication

UART communication is used for debugging, monitoring sensor values, and displaying system status messages through the Hercules serial terminal.

The project uses UART0 with a baud rate of 9600.

UART Settings

Parameter	Value
Baud Rate	9600
Data Bits	8
Stop Bits	1
Parity	None

Table 9

UART Initialization Code

```
LPC_UART0->LCR = 0x83;  
  
LPC_UART0->DLM = 0;  
  
LPC_UART0->DLL = 97;
```

```
LPC_UART0->LCR = 0x03;
```

Explanation

- LCR = 0x83 enables access to baud rate registers.
- DLL = 97 sets baud rate to 9600.
- LCR = 0x03 configures:
 - 8-bit data
 - 1 stop bit
 - no parity

UART TX Operation

UART transmission sends characters one by one through the TX pin.

UART TX Code

```
while (!(LPC_UART0->LSR & (1 << 5)));  
  
LPC_UART0->THR = *str++;
```

Explanation

- Bit 5 of **LSR** indicates transmitter ready.
- The program waits until UART is ready.
- The character is written into **THR**.

UART RX Operation

UART reception reads incoming serial data from the RX pin.

UART RX Code

```
if (LPC_UART0->LSR & 0x01) {  
  
    char c = LPC_UART0->RBR;  
}
```

Explanation

- Bit 0 of **LSR** indicates data received.
- **RBR** stores received data.

UART Output Example

```
SYSTEM READY  
AUTO MODE  
TEMP:1500|MOIST:2400|LIGHT:900
```

LED ON

6.3 Timer Interrupt

The project uses Timer1 interrupt to create a real-time clock system. The timer generates an interrupt every 1 second.

The interrupt updates the system time continuously.

Timer Configuration Code

```
LPC_TIM1->PR = 0;
```

```
LPC_TIM1->MR0 = SystemCoreClock - 1;
```

```
LPC_TIM1->MCR = 0x03;
```

Explanation

Register	Function
PR	Prescaler register
MR0	Match register
MCR	Match control register

Table 10

1-Second Interrupt

The timer interrupt occurs every second.

Timer Interrupt Handler

```
void TIMER1_IRQHandler(void) {  
    if (LPC_TIM1->IR & 0x01) {  
        LPC_TIM1->IR = 0x01;  
        update_time();  
    }  
}
```

Explanation

- Check interrupt flag

- Clear interrupt flag
- Update system clock

Real-Time Clock Update

The system time is updated in hours, minutes, and seconds.

Time Update Code

```
current_time.seconds++;  
  
if (current_time.seconds >= 60) {  
  
    current_time.seconds = 0;  
  
    current_time.minutes++;  
}
```

Clock Operation

- Seconds increment every interrupt
- Minutes increment every 60 seconds
- Hours increment every 60 minutes

7. Code

[Green house project full code](#)

END